

# ezr-w1.lua

ezr, week 1: columns, streaming, forgetting. One weekly-sized chunk of [ezr.lua](#), verbatim, with glossary notes folded in (click a ► to open one) and this week's exercises at the bottom.

## This week's story

A column watches values stream past and keeps a tiny summary: a Num holds mean and standard deviation, a Sym holds counts, mode and entropy. Nothing stores the values themselves. The stats also run backwards: subtract a value and they roll back, so a table can forget rows as cheaply as it learned them.

### ▼ noir

Nominal, Ordinal, Interval, Ratio (Stevens 1946): the four scales of measurement. This code collapses them to two: symbols you can only count (nominal) and numbers you can subtract (interval and up).

One header letter decides which:

```
-- Column kind from the first letter: lowercase makes a SYM,
-- uppercase a NUM.
function Col(name,at)
  return (name:find"^[%l]" and Sym or Num)(name,at) end
```

### ▼ Num

The summary of a numeric column: count `n`, mean `mu`, and `m2` (the sum of squared deviations from the mean, from which the standard deviation falls out). Nothing else is stored — not the data, just three numbers. A trailing `-` in the name means "goal: minimize".

```
function Num(name,at)
  name = name or ""
  return new(NUM, {at=at or 1, name=name, n=0, mu=0, m2=0,
    heaven = name:find"- $" and 0 or 1}) end
```

That last line, in Python, uses the True/False-is-1/0 trick (bools ARE ints, so arithmetic on a test needs no if):

```
heaven = 1 - (name[-1] == "-")  # True==1, False==0
```

### ▼ Sym

The summary of a symbolic column: count `n` and a table of counts `has`. Again, no data kept — just the histogram.

```
function Sym(name,at)
  return new(SYM, {at=at or 1, name=name or "", n=0, has={}}) end
```

```
function Col(name,at)
  return (name:find"^[%l]" and Sym or Num)(name,at) end
```

```
function Num(name,at)
  name = name or ""
  return new(NUM, {at=at or 1, name=name, n=0, mu=0, m2=0,
    heaven = name:find"- $" and 0 or 1}) end

function Sym(name,at)
  return new(SYM, {at=at or 1, name=name or "", n=0, has={}}) end
```

```
function SYM.add(i,v,inc)
  if v == "?" then return v end
  inc = inc or 1
  i.n = i.n + inc
  i.has[v] = inc + (i.has[v] or 0)
  if i.has[v] <= 0 then i.has[v] = nil end
  return v end

function NUM.add(i,v,inc, d)
  if v == "?" then return v end
  inc = inc or 1
  i.n = i.n + inc
  d = v - i.mu
  i.mu = i.mu + inc * d / i.n
  i.m2 = i.m2 + inc * d * (v - i.mu); return v end
```

▼ columnProtocol

Num and Sym answer the same eight questions — one polymorphic protocol, two implementations.

Everything downstream (tables, distance, trees, cuts) talks to the protocol, never to the type:

| question | Num answers                     | Sym answers                |
|----------|---------------------------------|----------------------------|
| add      | <u>welford</u> update of mu, m2 | bump a count in <u>has</u> |
| sub      | welford, run backwards          | drop a count               |
| mid      | <u>mean</u>                     | <u>mode</u>                |
| div      | <u>standard deviation</u>       | <u>entropy</u>             |
| norm     | <u>cdf</u> position, 0.1        | identity                   |
| dist     | gap of normed values            | 0 if same else 1           |
| holds    | x <= v                          | x == v                     |
| reset    | zero mu, m2                     | empty has                  |

Two samples, both sides of add:

```
function NUM.add(i,v,inc, d)
  if v == "?" then return v end
  inc = inc or 1
  i.n = i.n + inc
  d = v - i.mu
  i.mu = i.mu + inc * d / i.n
  i.m2 = i.m2 + inc * d * (v - i.mu); return v end

function SYM.add(i,v,inc)
  if v == "?" then return v end
  inc = inc or 1
  i.n = i.n + inc
  i.has[v] = inc + (i.has[v] or 0)
  if i.has[v] <= 0 then i.has[v] = nil end
  return v end
```

Note the shared conventions: "?" (missing) is ignored on the way in, and inc=-1 runs the summary backwards (see stream).

▼ welford

Welford's 1962 one-pass update: mean and variance from a stream, no stored data, no catastrophic cancellation. After each value *v*:

$$n' = n+1, \quad d = v - \mu, \quad \mu' = \mu + d/n', \quad m2' = m2 + d (v - \mu')$$

then  $sd = \sqrt{m2/(n-1)}$ . The NUM.add code above is exactly these four lines. Run with inc=-1 the algebra inverts, which is what makes summaries subtractable.

▼ mid (mode, mean)

The most frequent symbol: a Sym's answer to mid ("what is typical here?"). The **mean is the same question asked of numbers** — both are one value standing in for the whole column. That is why mid is one protocol slot, not two functions with different names:

```
function SYM.mid(i, hi,out)
  hi = -1
  for k, n in pairs(i.has) do
    if n > hi then hi, out = n, k end end
  return out end

function NUM.mid(i) return i.mu end
```

```
function SYM.mid(i, hi,out)
  hi = -1
  for k, n in pairs(i.has) do
    if n > hi then hi, out = n, k end end
  return out end

function NUM.mid(i) return i.mu end

function SYM.div(i)
  return sum(i.has, function(n, p)
    p = n / i.n
    return -p * log(p) / log(2) end) end

function NUM.div(i)
  return i.n < 2 and 0 or sqrt(max(i.m2,0) / (i.n-1)) end
```

### ▼ diversity (entropy, standard deviation)

Shannon 1948: the spread of a symbol column, in bits — the mean surprise of drawing from counts  $p_k$

$= n_k/n$ :

$$e = -\sum_k p_k \log_2 p_k$$

All-same symbols: 0 bits. Uniform over  $k$  symbols:  $\log_2 k$  bits. **Variance (or sd) is the same question**

**asked of numbers** — "how far is this column from settled?" — which is why **div** ("diversity") is one protocol slot with two spellings:

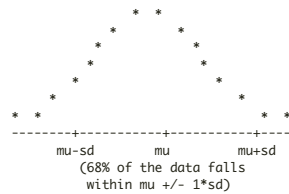
```
function SYM.div(i)
  return sum(i.has, function(n, p)
    p = n / i.n
    return -p * log(p) / log(2) end) end

function NUM.div(i)
  return i.n < 2 and 0 or sqrt(max(i.m2,0) / (i.n-1)) end
```

The two analogies are one design rule: every protocol slot names a question; each type answers in its own dialect. Central tendency: mean, mode. Diversity: sd, entropy. Trees built on **div** therefore handle numeric and symbolic goals with the same code.

### ▼ gaussian (mean, second moment)

The bell curve:



Summarized by two moments: the first moment  $\mu$  (the mean, **mu**) and, from the second moment, the spread (**m2** = sum of squared deviations, so  $sd = \sqrt{m2/(n-1)}$ ). Two tricks make these course-critical:

both update \*incrementally\*, one value at a time (**welford**); and two summaries \*subtract\* without resampling. From **ezr-eg1**'s **--without**: pour **{10,20,30}** into a summary of **{1,2,3,4,5}**; subtract a summary of **{10,20,30}**, and mu and sd of **{1..5}** come back exactly. Learn, unlearn, in  $O(1)$  — no stored data (see **stream**).

### ▼ cdf (cumulative distribution function)

The fraction of a population at or below a value: the area under the **pdf** up to  $x$ . Monotone, 0..1 — which makes it a natural normalizer: **norm** maps any cell to "what fraction of this column sits below you?". The normal cdf has no closed form, so **ezr** uses a logistic approximation (good to about  $\pm 1\%$ ), with the z-score clamped to  $\pm 3$ :

```
cdf(z) ≈ (1)/(1 + e^(-1.702 * z))

function NUM.norm(i,v, z)
  if v == "?" then return v end
  z = (v - i.mu) / (i.div() + TINY)
  return 1 / (1 + exp(-1.702 * max(-3, min(3, z)))) end
```

A bonus, for later: discretization rides on this for free. To turn any number into a small bin id, take **floor(the.bins \* num: norm(23))** — since norm is the cdf, equal-width slices of 0..1 give (roughly) equal-frequency bins of the data.

```
function NUM.norm(i,v, z)
  if v == "?" then return v end
  z = (v - i.mu) / (i.div() + TINY)
  return 1 / (1 + exp(-1.702 * max(-3, min(3, z)))) end
```

#### ▼ stream

A summary you can update — and un-update — one datum at a time, in constant memory. The `inc` argument (+1 or -1) means adding is  $O(1)$  and so is deleting, so any add-and-forget sweep over  $n$  items runs in linear time. Remember that: it matters later. Trees will score EVERY possible split of a sorted column in one linear pass — adding each row to the summary on one side of the cut while forgetting it from the other — where naive rebuilding would cost  $O(n^2)$ . It is also why `(a+b)-b == a` is a testable law (`--without`, `--sub` in `ezr-eg1`).

```
function NUM.reset(i) i.n, i.mu, i.m2 = 0, 0, 0 end
function SYM.reset(i) i.n, i.has = 0, {} end
```

## exercises

**\*\*Exercises, week 1\*\*** 1. Port this page's code to Python, wiring each demo of `[ezr-eg1](ezr-eg1.html)` to a `test_` function. 2. Add `{10,20,30}` to a `Num` one value at a time, printing `mu` and `sd` after each add. Watch Welford converge. 3. What is the entropy of a `Sym` fed "a","a","a"? Fed "a","b","c"? Predict first, then run it. 4. In `NUM.__sub`, why does `m2` get clamped with `max(0,...)`? What does that say about floats? 5. Flip `'heaven'` to `'and 1 or 0'`. Every test still passes. What breaks, and how would you ever notice?

```
function NUM.__sub(i,j, n,d)
  n = i.n - j.n
  if n < 1 then return Num(i.name, i.at) end
  d = j.mu - i.mu
  return new(NUM, {name=i.name, at=i.at, heaven=i.heaven,
                  n=n, mu=(i.n*i.mu - j.n*j.mu) / n,
                  m2=max(0, i.m2 - j.m2 - d*d*i.n*j.n/n)}) end

function NUM.reset(i) i.n, i.mu, i.m2 = 0, 0, 0 end
function SYM.reset(i) i.n, i.has = 0, {} end
```

```
local _ = "week 1 ends here"
```